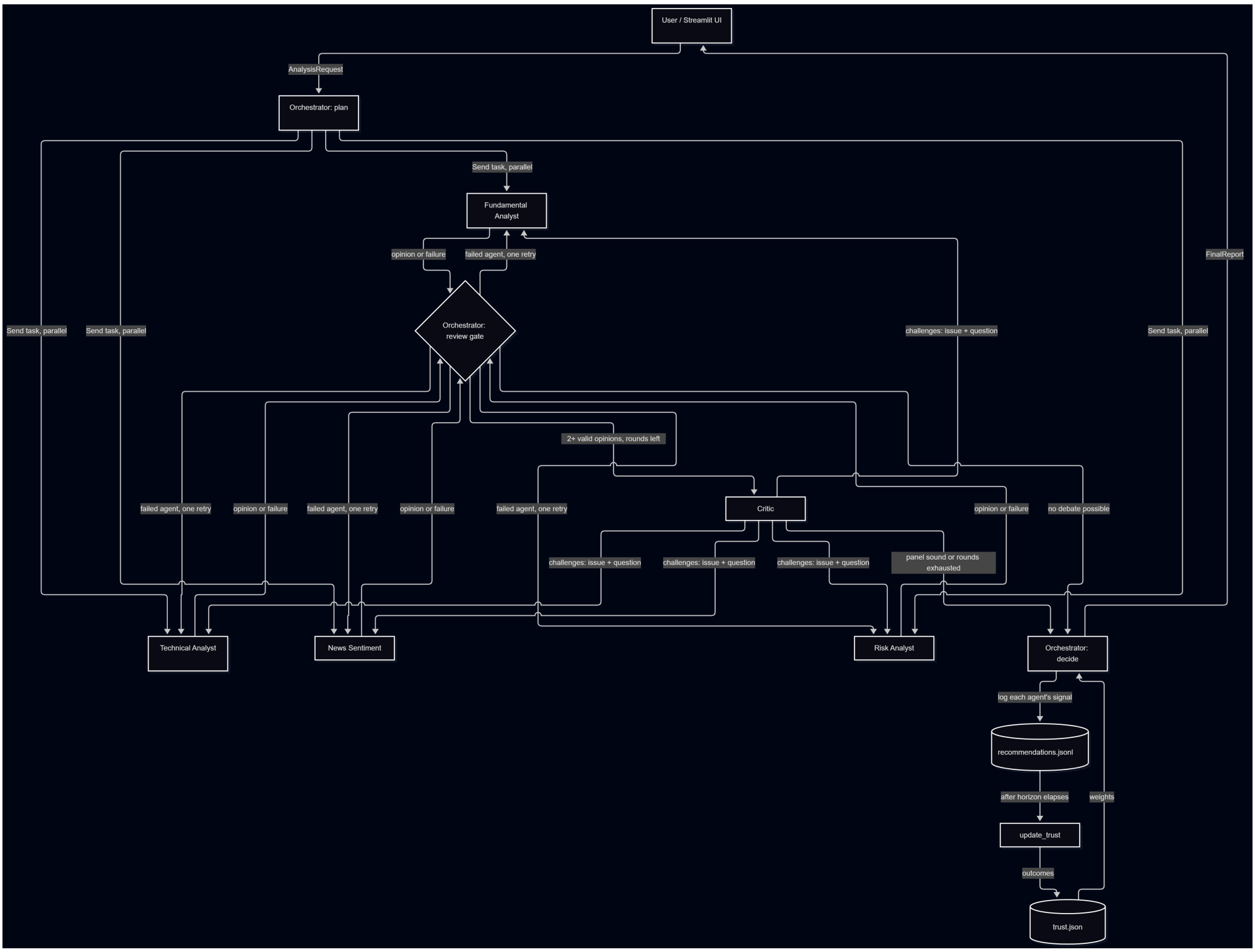
QuantCrew — Design Document

Team ZEUS. Export this file to PDF for the Design Document artifact. Render the Mermaid diagram at mermaid.live, export it as PNG, and place the image under section 1 in the PDF.

1. Architecture



All agents read and write one shared LangGraph state (the blackboard): opinions, errors, critiques, revisions and the execution trace. Requests flow down from the Orchestrator; opinions and failures flow up into the shared state; challenges flow from the Critic back to specific agents; market outcomes flow back into trust weights, closing the learning loop.

2. Agent roles

Agent	Responsibility	Decides for itself	Output
Orchestrator	Validates input, plans which specialists the question needs and the horizon, retries failed agents, triggers debate, computes the trust-weighted decision, writes the research note	Which agents to activate; horizon if unstated	Plan, FinalReport
Fundamental Analyst	Valuation, profitability, leverage, growth trend	Which figures matter for this sector	AgentOpinion
Technical Analyst	Trend, momentum, 52-week position	Whether to inspect the raw price path as well as indicators	AgentOpinion
News Sentiment	Tone and materiality of recent headlines	Which headlines are material	AgentOpinion
Risk Analyst	Volatility, drawdown, VaR/CVaR, beta, relative to the index	Whether risk is unusual for the horizon	AgentOpinion
Critic	Fact-checks claims with its own tools; challenges weak, unsupported or contradictory reasoning	Which agents to challenge, and whether revision is needed at all	CriticReview

Every specialist runs a ReAct loop (`agents/base.py`): the LLM chooses which tools to call, sees the results including errors, may call more tools, then commits to a structured `AgentOpinion`. If the model produces an invalid tool call, it is told so and allowed to try again.

Model choice per agent. All agents run on Groq's free tier: the Orchestrator and four specialists on gpt-oss-20b, the Critic on gpt-oss-120b. Specialists do narrow, tool-grounded work, so a fast model suffices; the Critic must verify other agents' numbers and spot contradictions, so it gets the strongest model. Groq's daily token limit is counted per model, so splitting the roles also doubles the daily budget. The provider is configurable (`QUANTCREW_PROFILE=claude` switches every role to Claude Haiku 5.5), but the submitted and deployed system uses the free tier.

3. Orchestration pattern

Hierarchical orchestrator–worker over a shared blackboard, with a critic-led debate loop.

- *Orchestrator–worker*: one agent plans and arbitrates; specialists run in parallel (LangGraph `Send`), at most two at a time to respect API rate limits.
- *Blackboard*: all agents write to one typed state (`state.py`) with merge reducers, so parallel writes don't collide and the Critic sees everyone's work.
- *Debate*: the Critic may challenge specific agents. All challenges for one agent are delivered together; the agent re-runs its tools and must revise or defend. One round by default (configurable); the loop ends early when the Critic judges the panel sound.
- *Replanning on failure*: an agent whose tools all fail, or whose LLM call errors, is retried once, then dropped, and the report is marked `degraded`. If an agent fails while revising, its earlier opinion is kept rather than discarded.

Conflict is detected when one agent says BUY and another SELL, when the panel is tied with no majority, or when a confident (≥ 0.6) agent disagrees with the majority.

Why not peer-to-peer: with five agents, all-to-all messaging is hard to bound and audit. A central arbiter keeps the number of LLM calls predictable and makes every decision traceable.

4. Technology choices

Choice	Reason
LangGraph	Native parallel fan-out (`Send`), conditional cycles for the debate loop, typed shared state with reducers, streaming updates for the UI
Custom ReAct loop instead of a prebuilt agent	Every tool call and thought is captured in the trace, and the team can explain every line
`init_chat_model`	One setting switches provider/model per role
Groq free tier (gpt-oss-20b / gpt-oss-120b)	Fast inference at no cost. The ~200k tokens/day limit per model shaped the design: one debate round, at most 3 tool steps per agent, at most 2 agents calling the API at once, no LLM planner or narrative in backtests, and a resumable backtest
Structured output with fallbacks	JSON-schema mode first, then function calling, then plain JSON validated by Pydantic; models differ in which they support reliably
yfinance + Google News RSS	Free NSE prices and fundamentals; Yahoo's NSE news is often empty, so Google News (last 7 days, searched by company name) is the fallback
Deterministic weighted vote for the final call	The LLM explains the decision but cannot overrule the evidence
Streamlit	Fast to deploy; shows each agent's progress live

5. Decision rule

Score = $\Sigma\, w_i \cdot c_i \cdot s_i \,/\, \Sigma\, w_i \cdot c_i$, where $s \in \{+1 \text{ BUY}, 0 \text{ HOLD}, -1 \text{ SELL}\}$, c = agent confidence, w = trust weight. BUY if score > 0.4, SELL if score < −0.4, otherwise HOLD. A BUY or SELL also requires that more than 50% of weighted conviction agrees with it; otherwise the call becomes HOLD (one confident agent cannot outvote a holding majority).

Confidence: for BUY/SELL, the share of weighted conviction agreeing with the call. For HOLD, 1 − (largest share of conviction pushing BUY or SELL). "Lean" reports which way the panel tilts.

6. Adaptive trust weights

Each agent's own signal is scored once its horizon has elapsed (5/20/60 trading days). Guards against noisy short-term outcomes:

1. **Dead band** — moves within ±2% are "no move": HOLD is correct; BUY/SELL are not scored either way.
2. **Beta(5,5) prior** — accuracy is the posterior mean, so a short lucky or unlucky streak barely moves it.
3. **Minimum sample size** — weight stays 1.0 until an agent has 15 scored calls.
4. **Rolling window** — only the last 60 scored calls count, so the system adapts to regime changes.
5. **Clamp** — weights stay within [0.5, 1.5]; no agent is silenced or dominant.

Weight = clamp(posterior accuracy / 0.5). Live calls are logged and scored by `scripts/update_trust.py` once their horizon has elapsed. In the submitted system every weight is still 1.0: no agent has yet reached 15 scored calls, which is the minimum-sample guard working as intended.

7. Evaluation

`scripts/backtest.py` walks forward through history, analysing a stock every 20 trading days with data up to that date only. The loop is date-major and feeds each call's outcome into trust only once it is known, so there is no look-ahead. Strategy: hold the stock for 20 days on BUY, otherwise cash.

Pilot run (INFY, Jul–Dec 2025, 6 decisions, 20-day horizon), within the free API budget:

Configuration	Return	Buy & hold	Sharpe	B&H Sharpe	Max DD	Time in market	Hit rate	Calls changed by debate
Full crew (Technical + Risk + Critic)	+9.0%	+5.4%	1.45	0.71	0.0%	17%	0.75	2
Technical agent only	+10.9%	+5.4%	1.76	0.71	0.0%	33%	1.00	—

Both beat buy-and-hold, but one +9% trade (25 Nov 2025, called BUY by both) drives most of the result, so six decisions support no statistical conclusion. The crew was more conservative: the Risk agent's caution turned a profitable BUY on 27 Oct into HOLD. The Risk agent's three scored calls were all wrong, so its posterior accuracy fell to 0.385; once it reaches 15 scored calls the trust mechanism would lower its weight to about 0.77. This is the adaptation the design intends.

The scripts also support the full study planned in the proposal (more stocks, `--no-debate`, `--agents risk`); it was not run because of the free tier's daily token limit.

Limitations: no point-in-time fundamentals or news in yfinance, so those agents are excluded from backtests; no transaction costs; LLMs may carry knowledge of the backtest period; LLM outputs vary between runs (temperature is 0, and the debate and weighted vote reduce but don't remove this).

8. Robustness

Situation	Behaviour
Symbol not in the Nifty 50	Rejected by the Orchestrator before any LLM call, with a clear message
No price data	Rejected with the data error
Empty question	UI asks for one
One agent's tools fail	The agent sees the error and may try another tool; if all fail, retried once, then excluded; report marked `degraded`
Agent fails while revising	Earlier opinion kept; `revision_failed` recorded in the trace
Invalid tool call by the model	Model told the call was invalid and retries
API rate limit	Client waits and retries; at most two agents call the API at once
Critic fails	Debate skipped, decision proceeds
Narrative LLM fails	Fallback note; recommendation unaffected (computed deterministically)

9. Issues found by reviewing execution traces

Trace showed	Fix
Fundamental agent quoting sector averages no tool provides	All agents and the Critic forbidden from using figures from memory
Debt-to-equity 9.5 read as 9.5× (yfinance gives a percentage)	Converted to a ratio; units note attached to tool results
Revenue in USD next to market cap in INR	Statement currency attached; agents told never to divide one by the other
Only the last of several challenges reached an agent	All challenges merged and delivered
HOLD-vs-SELL tie not flagged as a conflict	Ties count as conflicts
One confident BUY outvoting two HOLDs	Majority-conviction rule
HOLD reported with 0% confidence	HOLD confidence and lean defined
Rate limit during revision discarded valid opinions	Earlier opinion kept
Deployed run invented historical margins and peer ranges	Prompts restrict figures to tool results; annual_trend contents stated explicitly